43008: Reinforcement Learning

Assignment - 2

Student Name: Warit Srichairattanakul

Student ID: 25745973

Selected Environment:

1. **Breakout Gymnasium Atari Environment**
2. **CartPole Gymnasium Classic Control**
3. **Pendulum Gymnasium Environment**

Introduction:

This report applies different Reinforcement Learning methods to solve three control and decision-making problems using Gymnasium environments. The goal is to understand how Temporal Difference learning, Deep Q-Learning, and Policy Gradient methods make the agent able to learn suitable actions through interaction with an environment. The experiment also compares the performance of different algorithms and examines how the characteristics of each environment affect the learning process.

There are three environments that have been used in this assignment. The first environment is CartPole, where the agent has to move a cart to keep the pole balanced on the cart as long as possible. The second environment is Breakout, which is an Atari game where the agent has to control a paddle to bounce a ball and destroy bricks to earn rewards. The third environment is Pendulum, where the agent applies torque force to a pendulum to swing it upwards and maintain it in upright position.

Deep Q-Network (DQN) and Proximal Policy Optimization (PPO) are implemented for Breakout and Cartpole. DQN is a value-based method that uses a neural network to estimate the expected return of different actions, while PPO is a policy gradient method that learns a policy through repeated interaction with the environment. For Pendulum, Tabular Q-learning is implemented as a Temporal Difference (TD) method. Since the Pendulum environment has continuous observation and action spaces, both spaces are discretized so that a Q-table can be used to learn the action values.

The agents are trained with multiple episodes or timesteps and evaluated using learning curves, average rewards, and simulation results. The experiments for Breakout and CartPole compare DQN and PPO, while the experiment for Pendulum examines the learning behaviour and limitations of Tabular Q-learning in a continuous control environment. The results are used to discuss the strengths and weaknesses of each method, the stability of the training process, and the suitability of different reinforcement learning approaches for different types of problems.

Scenario and Algorithm

Scenario 1: Breakout (Atari Environment)

Breakout is an Atari game where the agent controls a paddle at the bottom of the screen to move right or left to the desired position. A ball moves around the environment and destroys bricks when it hits them. The agent must move the paddle to prevent the ball from falling below the screen while trying to destroy as many bricks as possible. The main aim is to learn a policy that can control the paddle based on the position and movement of the ball.

States

The state of the Breakout environment is represented by visual observations from the game screen. In this experiment, the observation space is defined as Box(0, 255, (84, 84, 4), uint8). Each observation contains four consecutive frames with a resolution of 84 x 84 pixels. The pixel values range from 0 to 255.

The four frames provide information about the position and movement of the ball, paddle, and remaining bricks. Using multiple frames allows the agent to understand the direction of the ball's movement, which may not be clear from a single image. A Convolutional Neural Network (CNN) is used to extract important features from these observations and support the decision-making process.

Actions

The Breakout environment has a discrete action space defined as Discrete(4). This means that the agent can select one of four possible actions at each timestep. These actions are NOOP, FIRE, RIGHT, and LEFT. NOOP performs no action, while FIRE is used to launch the ball. RIGHT and LEFT allow the agent to move the paddle in the corresponding direction. The agent must learn when to move the paddle to prevent the ball from falling and increase the chance of destroying bricks. Since the action space is discrete, both DQN and PPO can be applied to learn suitable actions based on the current observation.

Reward

The agent receives positive rewards when the ball hits and destroys bricks. The reward depends on the bricks destroyed during gameplay. When the ball moves without destroying any bricks, the agent generally receives a reward of zero.

The reward structure encourages the agent to move the paddle to keep the ball in play and destroy as many bricks as possible. However, the agent does not receive an immediate positive reward for every suitable paddle movement. For example, moving the paddle to catch the ball may prevent it from falling, but the agent may only receive a positive reward later when the ball destroys a brick. This makes the learning process more difficult because the agent must learn how its previous actions contribute to future rewards.

Goal

The main goal of the Breakout environment is to maximise the total reward by destroying as many bricks as possible while preventing the ball from falling below the paddle. To achieve this, the agent must learn how to track the movement of the ball and move the paddle to a suitable position.

An agent should be able to keep the ball in play for longer periods and consistently destroy bricks to increase its total score. In this experiment, DQN and PPO are used to learn suitable policies for controlling the paddle. Their performance is evaluated and compared using training rewards, learning curves, and gameplay simulations.

Algorithm 1: Deep Q-Network (DQN)

Deep Q-Network (DQN) is a value-based reinforcement learning algorithm that uses a neural network to estimate the value of different actions. Unlike traditional Q-learning, which stores action values in a Q-table, DQN uses a neural network to predict how much reward the agent may receive in the future. The agent then uses these predictions to decide which action to take.

In this experiment, DQN is implemented using Stable-Baselines3 with a custom Convolutional Neural Network (CNN). The CNN has three convolutional layers with 32, 64, and 64 filters, followed by a fully connected layer with 512 features. It helps the agent identify important information from the game images, such as the position and movement of the ball. The experiment uses the BreakoutNoFrameskip-v4 environment with four parallel environments and four stacked frames.

During training, DQN uses an epsilon-greedy strategy to explore different actions. At the beginning, the agent takes more random actions to learn about the environment. As training continues, it gradually selects actions based on what it has learned. DQN also stores previous experiences in a replay buffer and uses them to improve the model. A separate target network is used to make learning more stable. In this experiment, the learning rate is 0.0001, the replay buffer size is 10,000, and the batch size is 32.

DQN was selected for Breakout because the game has a limited number of actions, and the agent needs to learn how to control the paddle from images. The CNN helps the agent understand the game screen, while DQN learns which actions are likely to give higher rewards. However, learning can take time because the agent does not receive a reward for every action. For example, moving the paddle to catch the ball may be useful, but the agent only receives a positive reward when the ball destroys a brick.

Algorithm 2: Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a policy gradient reinforcement learning algorithm that learns how to select actions based on the current state. Instead of predicting the value of each action like DQN, PPO learns which actions are more likely to lead to better results. It uses two parts: an actor that selects actions and a critic that estimates how good the current state is.

In this experiment, PPO is implemented using Stable-Baselines3 with a custom CNN. The CNN has the same structure as the one used for DQN, with three convolutional layers containing 32, 64, and 64 filters, followed by a fully connected layer with 512 features. PPO also uses the BreakoutNoFrameskip-v4 environment with four parallel environments and four stacked frames. Using the same environment and CNN structure makes it easier to compare the two algorithms.

During training, PPO collects experiences by playing the game and uses them to improve its policy. One important feature of PPO is the clipping range, which helps prevent the policy from changing too much during training. The clipping range controls how much the probability ratio between the new and old policies can change before the PPO objective is clipped. In this experiment, the clipping range is set to 0.1, which means that the ratio is clipped to the range of 0.9 to 1.1. This helps reduce large policy updates and makes the learning process more stable. The learning rate is set to 0.00025, and the model performs four optimisation epochs for each rollout. An entropy coefficient of 0.01 is also used to encourage the agent to explore different actions.

PPO was selected for Breakout because it can learn how to control the paddle directly from game images. The CNN helps the agent understand the game screen, while PPO improves its action selection based on the experiences collected during training. Unlike DQN, which can reuse older experiences stored in a replay buffer, PPO mainly learns from experiences collected using its current policy. This allows the experiment to compare two different learning approaches and examine how well they perform in the same environment.

Scenario 2: CartPole (Classic Control Environment)

CartPole is a classic control environment where a pole is attached to a cart that can move left or right along a horizontal track. The agent controls the movement of the cart to keep the pole balanced in an upright position. The agent must select suitable actions to prevent the pole from falling while keeping the cart within the track boundaries. The main aim is to learn a policy that can maintain balance based on the position and movement of the cart and pole.

States

The state of the CartPole environment is represented by four numerical observations. In this experiment, the observation space is a continuous Box space with a shape of (4,). The four values are cart position, cart velocity, pole angle, and pole angular velocity.

The cart position and velocity describe where the cart is and how it is moving. The pole angle shows how far the pole is leaning from the upright position, while the angular velocity describes how quickly the pole is rotating. These observations help the agent understand the current condition of the system and decide which direction to move the cart.

Unlike Breakout, CartPole does not require image processing because the observations are already provided as numerical values. An Artificial Neural Network (ANN) is used to process these observations and learn the relationship between the current state and suitable actions.

Actions

The CartPole environment has a discrete action space defined as Discrete(2). This means that the agent can select one of two possible actions at each timestep. Action 0 applies a force to move the cart to the left, while Action 1 applies a force to move the cart to the right.

The agent must learn when to move left or right based on the pole angle and movement. For example, if the pole is falling towards one side, the agent may need to move the cart in that direction to maintain balance. Since the action space is discrete, both DQN and PPO can be used to learn suitable actions based on the current observation.

Reward

In CartPole-v1, the agent receives a reward of +1 for each timestep, including the final timestep when the episode ends. The reward encourages the agent to keep the pole balanced for as long as possible. If the pole falls beyond the allowed angle or the cart moves outside the track boundaries, the episode ends.

The maximum episode length is 500 timesteps, so the highest possible total reward in one episode is 500. Unlike Breakout, where the agent receives rewards mainly when bricks are destroyed, CartPole provides a reward at every timestep. This allows the agent to receive more frequent feedback about how long it can maintain balance.

Goal

The main goal of the CartPole environment is to maximise the total reward by keeping the pole upright and preventing the cart from moving outside the allowed boundaries. To achieve this, the agent must learn how to respond to changes in the pole angle, angular velocity, and cart movement.

An agent should be able to maintain balance for longer periods and consistently achieve higher episode rewards. In this experiment, DQN and PPO are used to learn suitable policies for controlling the cart. Their performance is evaluated and compared using training rewards, learning curves, and simulation results.

Algorithm 1: Deep Q-Network (DQN)

Deep Q-Network (DQN) is a value-based reinforcement learning algorithm that uses a neural network to estimate the value of different actions. In CartPole, DQN estimates the expected future reward of moving the cart left or right. The agent uses these estimated Q-values to select actions that are likely to keep the pole balanced for longer periods.

In this experiment, DQN is implemented using Stable-Baselines3 with a custom Artificial Neural Network (ANN). The ANN receives four observation values as input and processes them through two fully connected layers. The first layer contains 128 neurons, while the second layer produces 64 features. These features are then passed to the DQN network head, which contains two additional hidden layers with 128 neurons each. The

network uses these features to estimate the Q-values of the two available actions. The experiment uses the CartPole-v1 environment, with the same feature extractor architecture as PPO to allow a more consistent comparison between the two algorithms.

During training, DQN uses an epsilon-greedy strategy to explore different actions. At the beginning, the agent takes more random actions to understand how moving the cart affects the pole. As training continues, it gradually selects actions based on the learned Q-values. The agent also stores previous experiences in a replay buffer and uses them to improve the model. In this experiment, the learning rate is 0.0001, the replay buffer size is 50,000, and the batch size is 128. The discount factor is set to 0.99 to consider future rewards when selecting actions.

DQN was selected for CartPole because the environment has only two actions and provides numerical observations that can be processed using an ANN. The model learns which direction to move the cart based on the current position and movement of the pole. However, the agent may still select unsuitable actions during exploration, which can cause the pole to fall and end the episode early.

Algorithm 2: Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a policy gradient reinforcement learning algorithm that learns how to select actions based on the current state. In CartPole, PPO learns the probability of moving the cart left or right to maintain balance. It uses an actor to select actions and a critic to estimate how good the current state is.

In this experiment, PPO is implemented using Stable-Baselines3 with a custom ANN. The ANN receives four observation values and processes them through two fully connected layers. The first layer contains 128 neurons, while the second layer produces 64 features. These features are then passed to the policy and value networks, which each contain two additional hidden layers with 128 neurons. The policy network uses these features to select actions, while the value network estimates the value of the current state. The experiment uses the same CartPole-v1 environment as DQN, allowing the two algorithms to be compared under the same control problem.

During training, PPO collects experiences by interacting with the environment and uses them to improve its policy. The algorithm uses a clipping mechanism to reduce large changes in the policy during each update. In this experiment, the learning rate is set to 0.0003, and the model collects 2,048 steps before each update. The batch size is 64, and the model performs 10 optimisation epochs for each rollout. An entropy coefficient of 0.01 is also used to encourage the agent to explore different actions.

PPO was selected for CartPole because it can learn how to control the cart using numerical observations and improve its policy through repeated interactions. The ANN helps the model understand the relationship between the cart's movement and the pole's balance. Unlike DQN, PPO updates its policy using recently collected experiences rather than a replay buffer. This allows the experiment to compare how the two algorithms learn to maintain balance in the same environment.

Scenario 3: Pendulum (Classic Control Environment)

Pendulum is a classic control environment where a pendulum is attached to a fixed point and can rotate freely. The agent controls the pendulum by applying torque to make it swing in different directions. The agent must learn how much torque to apply to bring the pendulum towards the upright position and keep it balanced. The main aim is to learn a policy that can control the pendulum based on its current angle and movement.

States

The state of the Pendulum environment is represented by three numerical observations. In this experiment, the environment uses Pendulum-v1, where the observation contains the cosine of the pendulum angle, the sine of the pendulum angle, and the angular velocity. These values describe the current position of the pendulum and how quickly it is rotating.

Unlike CartPole, Pendulum has a continuous observation space. Since Tabular Q-learning requires discrete states, the observations are converted into a smaller number of states. The pendulum angle is calculated from the sine and cosine values, while the angular velocity is taken directly from the observation.

In this experiment, the angle range from -π to π is divided into 30 bins, and the angular velocity range from -8 to 8 is also divided into 30 bins. This gives a total of 900 discrete states. Each observation is assigned to one of these states so that the agent can store and update action values in a Q-table.

Actions

The Pendulum environment has a continuous action space where the agent can apply torque between -2 and +2. Negative torque applies force in one direction, while positive torque applies force in the opposite direction. The amount of torque affects how quickly the pendulum rotates.

Since Tabular Q-learning requires a finite number of actions, the continuous torque range is divided into nine discrete actions. These actions are -2, -1.5, -1, -0.5, 0, 0.5, 1, 1.5, and 2.

The agent selects one of these nine actions based on the current discrete state. For example, it may apply a larger torque to swing the pendulum upwards or a smaller torque to control its movement when it is close to the upright position. This allows Tabular Q-learning to be applied to the continuous control problem.

Reward

The Pendulum environment uses a negative reward function based on the pendulum angle, angular velocity, and applied torque. The reward is calculated as:

$$Reward = -(\theta_t^2 + 0.1\omega_t^2 + 0.001u_t^2)$$

where θ is the pendulum angle, ω is the angular velocity, and u is the applied torque.

The agent receives a larger penalty when the pendulum is far from the upright position, rotates too quickly, or uses excessive torque. Unlike Breakout and CartPole, Pendulum does not provide positive rewards for successful actions. Instead, a reward closer to zero indicates better performance.

The reward structure encourages the agent to keep the pendulum upright while reducing unnecessary movement and control effort. For example, applying a large torque may help the pendulum swing upwards, but it also increases the torque penalty. The agent must therefore learn how to balance these factors to achieve a higher total reward.

Goal

The main goal of the Pendulum environment is to maximise the total reward by swinging the pendulum upwards and keeping it as close as possible to the upright position. To achieve this, the agent must learn how to apply suitable torque based on the pendulum angle and angular velocity.

An agent should be able to reduce large movements and maintain a more stable position over time. In this experiment, Tabular Q-learning is used to learn a policy for controlling the pendulum. Its performance is evaluated using training rewards, learning curves, and gameplay simulations at different training stages.

Algorithm: Tabular Q-learning

Tabular Q-learning is a value-based reinforcement learning algorithm that uses Temporal Difference (TD) learning to estimate the value of different actions. The algorithm stores the expected future reward of each state-action pair in a Q-table. The agent uses these values to select actions and updates the table based on the rewards received from the environment.

In this experiment, Tabular Q-learning is implemented with the Pendulum-v1 environment. Since the environment has continuous states and actions, both are converted into discrete values. The Q-table contains 30 angle bins, 30 angular velocity bins, and nine torque actions, giving a total of 8,100 Q-values. Each value represents the expected future reward of applying a particular torque in a given state.

During training, the agent uses an epsilon-greedy strategy to balance exploration and exploitation. At the beginning, epsilon is set to 1, so the agent explores different torque actions. As training continues, epsilon gradually decreases to a minimum of 0.05, allowing the agent to select actions based more often on the learned Q-values. After each action, the Q-table is updated using the reward received and the highest estimated Q-value of the next

state. In this experiment, the learning rate is set to 0.1, the discount factor is 0.99, and the agent is trained for 20,000 episodes.

Tabular Q-learning was selected for Pendulum because it provides a simple way to study how an agent can learn from rewards without using a neural network. However, converting continuous values into discrete states and actions also has limitations. For example, different pendulum angles may be grouped into the same state, and the agent can only select from nine torque values. This may make it difficult to control the pendulum precisely when it is close to the upright position. The experiment therefore examines how the agent improves during training and how discretization affects its performance.

Diagram

Deep Q-Network (DQN)

Flowchart

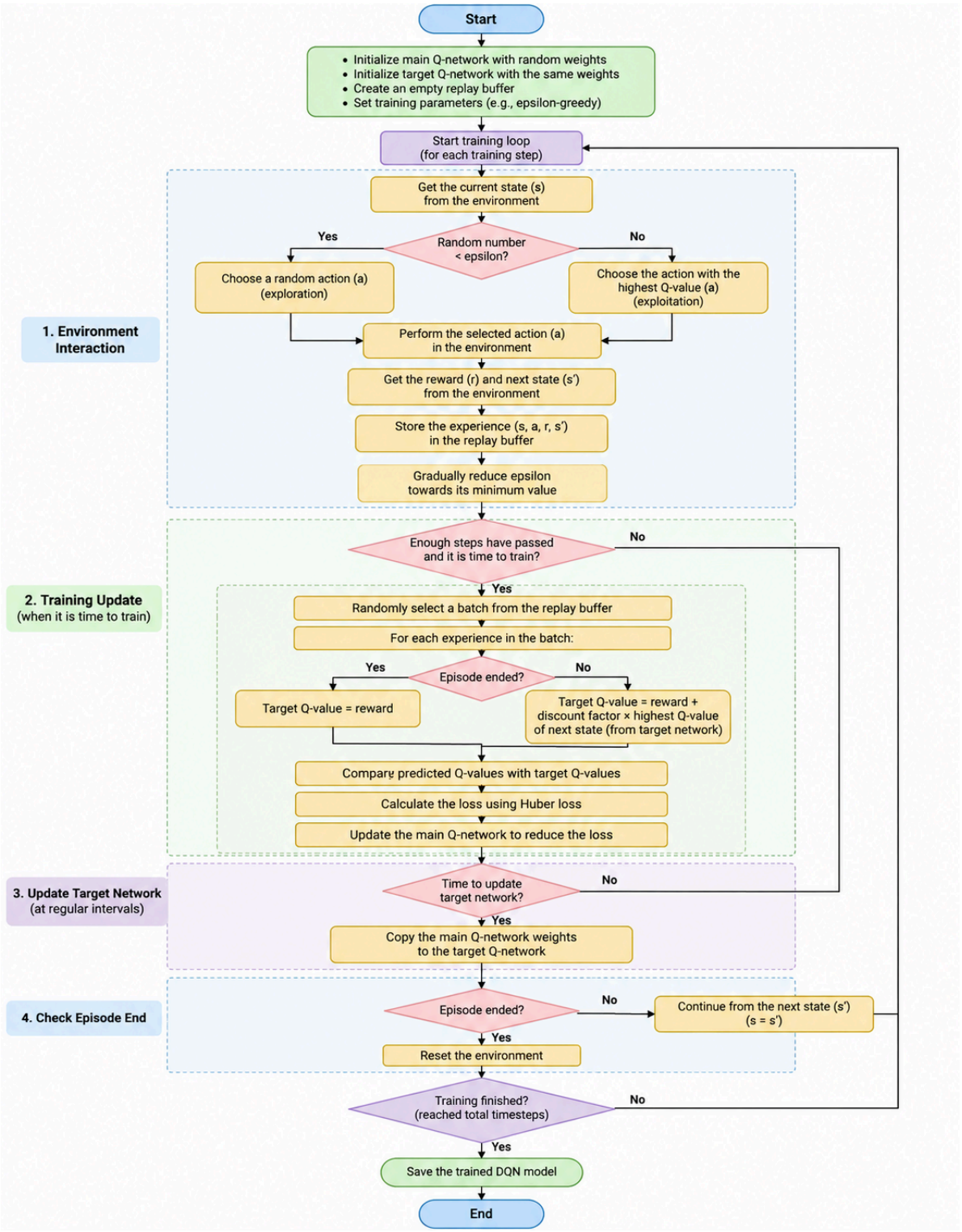


Figure 1. Deep Q-Network algorithm flowchart

Pseudocode

```
Initialize the main Q-network with random weights
Initialize the target Q-network with the same weights as the main Q-network
Create an empty replay buffer
Set the training parameters, e.g., epsilon-greedy

For each training step:
    Get the current state (s) from the environment
    Choose an action (a) using epsilon-greedy:
        If a random number is less than epsilon:
            Choose a random action (exploration)
        Else:
            Choose the action with the highest Q-value (exploitation; based on
        the current Q-network)

    Perform the selected action (a) in the environment
    Get the reward (r) and next state (s') from the environment
    Store the experience (s, a, r, s') in the replay buffer

    Gradually reduce epsilon towards its minimum value

    If enough steps have passed and it is time to train:
        Randomly select a batch from the replay buffer

        For each experience in the batch:
            If the episode has ended:
                Target Q-value = reward
            Else:
                Target Q-value = reward + discount factor * highest Q-value
            of the next state      predicted by the target network
        Compare the predicted Q-values with target Q-values
        Calculate the loss using Huber loss
        Update the main Q-network to reduce the loss

    At regular intervals:
        Copy the main Q-network weights to the target Q-network

        If the episode has ended:
            Reset the environment
        Else:
            Continue from the next state
Save the trained DQN model
```

Proximal Policy Optimization (PPO)

Flowchart

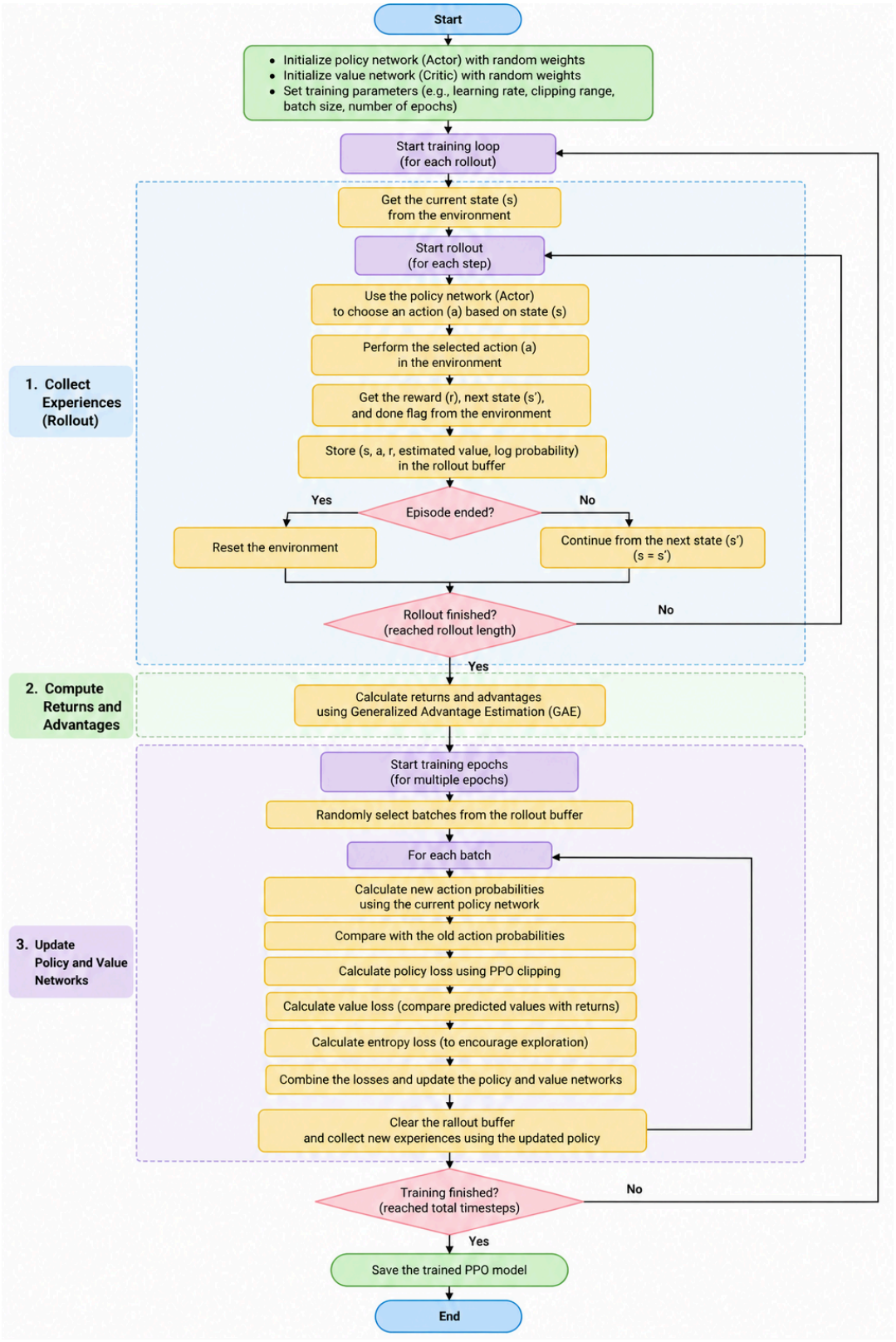


Figure 2. Proximal Policy Optimization algorithm flowchart

Pseudocode

```
Initialize the policy network (Actor) with random weights
Initialize the value network (Critic) with random weights
Set the training parameters, e.g., clipping range

For each training rollout:
    Get the current state (s) from the environment

    For each step in the rollout:
        Use the policy network to choose an action (a) based on the current state (s)
        Perform the selected action (a) in the environment
        Get the reward (r), next state (s'), and done flag

        Store the state (s), action (a), reward (r), estimated state value, and action
    log probability in the rollout buffer

        If the episode has ended:
            Reset the environment
        Else:
            Continue from the next state (s')

    Calculate the returns and advantages using Generalized Advantage Estimation

    For each training epoch:
        Randomly select batches from the rollout buffer

    For each batch:
        Calculate the new action probabilities using the current policy network
        Compare the new action probabilities with the probabilities from the old policy
        Calculate the policy loss using PPO clipping to prevent large policy updates
        Calculate the value loss by comparing predicted state values with the returns
        Calculate the entropy loss to encourage exploration
        Combine the losses and update the policy and value networks

Clear the old rollout data and collect new experiences using the updated policy

Repeat until the total training timesteps are reached
```

Tabular Q-learning

Flowchart

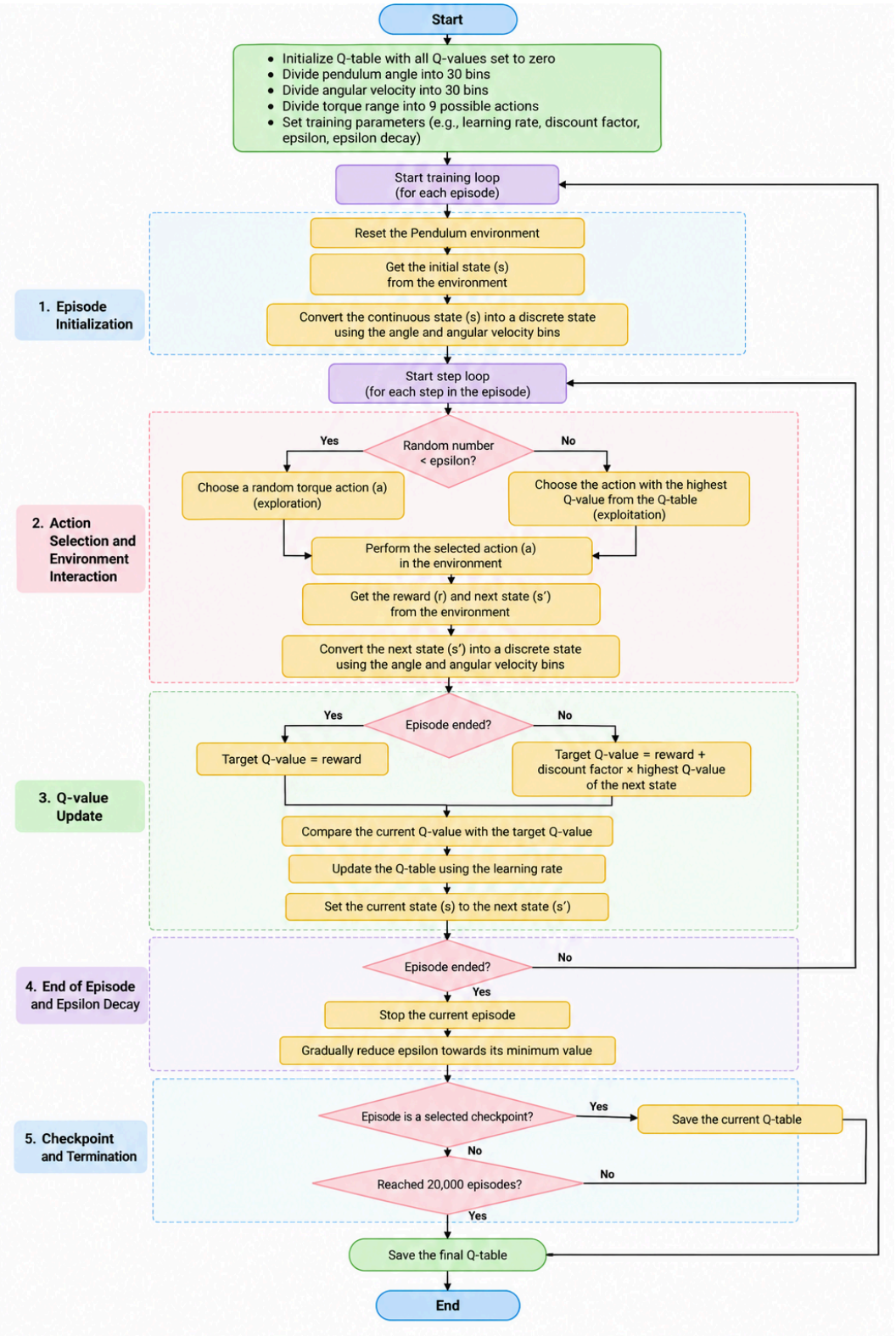


Figure 3. Tabular Q-learning algorithm flowchart

Pseudocode

```
Initialize the Q-table with all Q-values set to zero
Divide the pendulum angle into 30 bins
Divide the angular velocity into 30 bins
Divide the torque range into 9 possible actions
Set the training parameters, e.g., discount factor, epsilon, and epsilon decay

For each training episode:
    Reset the Pendulum environment
    Get the initial state (s) from the environment

    Convert the continuous state (s) into a discrete state using the angle and angular velocity bins

    For each step in the episode:
        Choose an action (a) using epsilon-greedy:
            If a random number is less than epsilon:
                Choose a random torque action (exploration)
            Else:
                Choose the action with the highest Q-value from the Q-table (exploitation)
        Perform the selected torque action (a) in the environment
        Get the reward (r) and next state (s')
        Convert the next state (s') into a discrete state

        If the episode has ended:
            Target Q-value = reward
        Else:
            Target Q-value = reward + discount factor * highest Q-value of the next state

        Compare the current Q-value with the target Q-value
        Update the Q-table using the learning rate
        Set the current state (s) to the next state (s')

        If the episode has ended:
            Stop the current episode
    Gradually reduce epsilon towards its minimum value

    If the episode is a selected checkpoint:
        Save the current Q-table

Repeat until 20,000 episodes are completed
Save the final Q-table
```

Neural Network Architectures

Breakout CNN

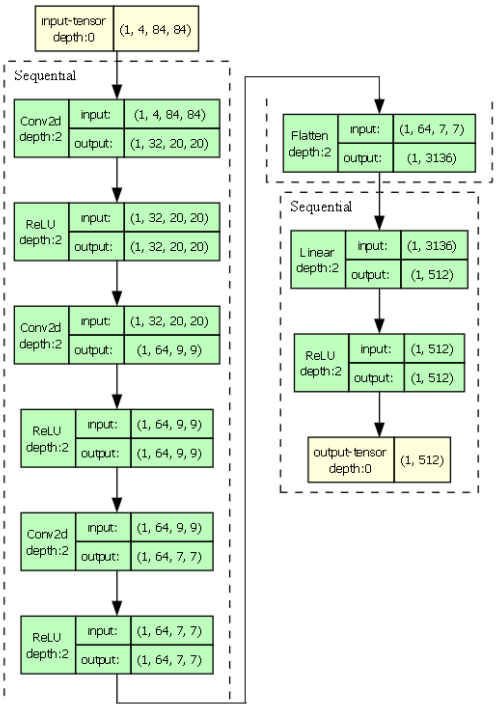


Figure 4. Architecture of the custom CNN feature extractor

CartPole ANN

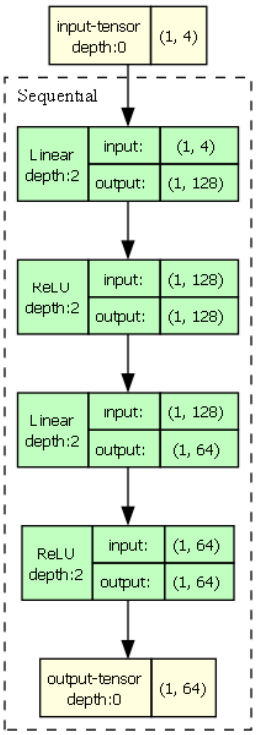


Figure 5. Architecture of the custom ANN feature extractor

Experimental results and discussion

Experimental Settings

Breakout

For this experiment, DQN and PPO are implemented using Stable-Baselines3 with the BreakoutNoFrameskip-v4 environment. Both algorithms use a custom Convolutional Neural Network (CNN) to process the game observations. The CNN contains three convolutional layers with 32, 64, and 64 filters, followed by a fully connected layer that produces 512 features. The observations are resized to 84 x 84 pixels, and four consecutive frames are stacked together to provide information about the movement of the ball.

Both algorithms are configured to train for 10 million timesteps. However, different hyperparameters are used because DQN and PPO learn using different methods. DQN uses a replay buffer and an epsilon-greedy exploration strategy, while PPO collects rollouts and updates its policy using a clipped objective.

For DQN, the replay buffer size is set to 10,000 to store previous experiences for training. Learning starts is set to 100,000, meaning that the model begins learning after collecting 100,000 timesteps. The exploration fraction is set to 0.1, so epsilon gradually decreases during the first 10% of training. The final epsilon is 0.01, meaning that the agent still has a 1% chance of selecting a random action.

Parameter	Value
Environment	BreakoutNoFrameskip-v4
Learning Rate	0.0001
Batch Size	32
Training Timesteps	10,000,000
Replay Buffer Size	10,000
Learning Starts	100,000
Exploration Fraction	0.1
Final Epsilon	0.01

Table 1. Breakout experimental settings for DQN

For PPO, the rollout steps are set to 128, meaning that the agent collects 128 steps from each environment before updating the policy. The optimisation epochs are set to 4, so the model trains on the collected data four times. The clipping range is 0.1 to help prevent large policy updates, while the entropy coefficient of 0.01 encourages the agent to explore different actions.

Parameter	Value
Environment	BreakoutNoFrameskip-v4
Learning Rate	0.00025
Batch Size	256
Training Timesteps	10,000,000
Rollout Steps	128
Optimisation Epochs	4
Clipping Range	0.1
Entropy Coefficient	0.01

Table 2. Breakout experimental settings for PPO

Cartpole

In the CartPole experiment, DQN and PPO are also implemented using Stable-Baselines3 with the CartPole-v1 environment. CartPole provides four numerical observations, so a custom Artificial Neural Network (ANN) is used instead of a CNN in Breakout.

Both algorithms are configured to train for one million timesteps. The DQN and PPO models use different learning rates and batch sizes because their training methods are different. DQN uses an epsilon-greedy strategy and a replay buffer to learn the Q-values of the two actions. In contrast, PPO collects experiences from the current policy and uses multiple optimisation epochs to improve its action selection.

For DQN, the replay buffer size is set to 50,000 to store previous experiences for training. Learning starts is set to 1,000, meaning that the model begins learning after collecting 1,000 timesteps. The exploration fraction is 0.2, so epsilon gradually decreases during the first 20% of training. The final epsilon is 0.05, meaning that the agent still has a 5% chance of selecting a random action. The discount factor of 0.99 allows the agent to consider future rewards when selecting actions.

Parameter	Value
Environment	CartPole-v1
Learning Rate	0.0001
Batch Size	128
Training Timesteps	1,000,000
Replay Buffer Size	50,000

Learning Starts	1,000
Exploration Fraction	0.2
Final Epsilon	0.05
Discount Factor	0.99

Table 3. CartPole experimental settings for DQN

For PPO, the rollout steps are set to 2,048, meaning that the agent collects 2,048 steps before updating its policy. The optimisation epochs are set to 10, so the model trains on the collected data ten times. The entropy coefficient of 0.01 encourages the agent to explore different actions, while the discount factor of 0.99 helps the agent consider future rewards.

Parameter	Value
Environment	CartPole-v1
Learning Rate	0.0003
Batch Size	64
Training Timesteps	1,000,000
Discount Factor	0.99
Rollout Steps	2048
Optimisation Epochs	10
Entropy Coefficient	0.01

Table 4. CartPole experimental settings for PPO

Pendulum

In the Pendulum experiment, Tabular Q-learning is implemented using NumPy with the Pendulum-v1 environment. Unlike DQN and PPO, this algorithm does not use a neural network. Instead, it stores the estimated action values in a Q-table.

Since Pendulum has continuous states and actions, the pendulum angle and angular velocity are each divided into 30 bins, while the torque range is divided into nine discrete actions. This produces a Q-table containing 8,100 state-action values, allowing the agent to learn which torque action is suitable for each discrete state.

The agent is trained for 20,000 episodes using a learning rate of 0.1 and a discount factor of 0.99 to consider future rewards. The initial epsilon is set to 1.0, meaning that the agent starts by selecting random actions to explore the environment. During training, epsilon decreases

exponentially to a minimum of 0.05, allowing the agent to gradually rely more on the learned Q-values when selecting actions.

Parameter	Value
Environment	Pendulum-v1
Angle Bins	30
Angular Velocity Bins	30
Discrete Torque Actions	9
Learning Rate	0.1
Discount Factor	0.99
Initial Epsilon	1.0
Minimum Epsilon	0.05
Epsilon Decay	Exponential

Table 5. Pendulum experimental settings for Tabular Q-learning

Experimental Results

Training Details

Breakout

In the Breakout experiment, both DQN and PPO were trained using the same game environment and custom CNN architecture. The models were configured to train for 10 million timesteps. The training rewards were recorded and plotted to observe how the agents improved during training.

For DQN, Figure 6 shows that the episode reward was close to zero during the first 3 million timesteps. After that, the reward gradually increased and reached around 150 at 8 million timesteps. During the final stage, the reward continued to increase and reached approximately 300. This shows that the agent improved its ability to play Breakout as training progressed. However, the reward also became more unstable during the later stages, and the curve was still increasing at the end of training.

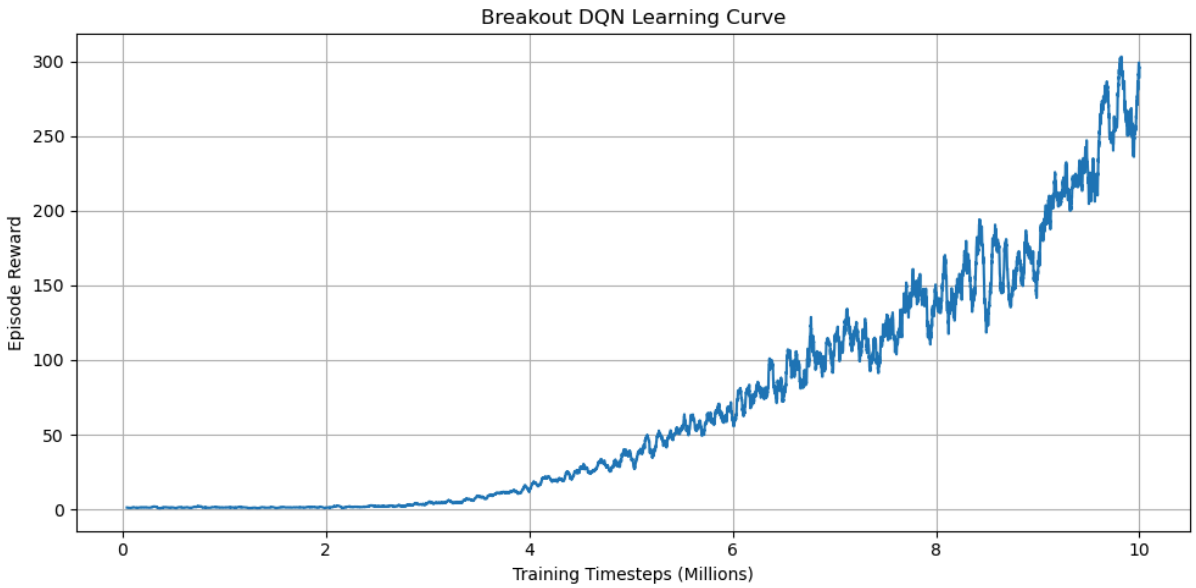


Figure 6. Breakout DQN Learning Curve

For PPO, Figure 7 shows that the reward started increasing earlier than DQN. During the first 3 million timesteps, PPO reached around 20 to 30 reward, while DQN was still close to zero. The reward continued to increase and reached around 150 at 8 million timesteps. During the final stage, PPO reached a maximum of approximately 290, but the reward fluctuated significantly.

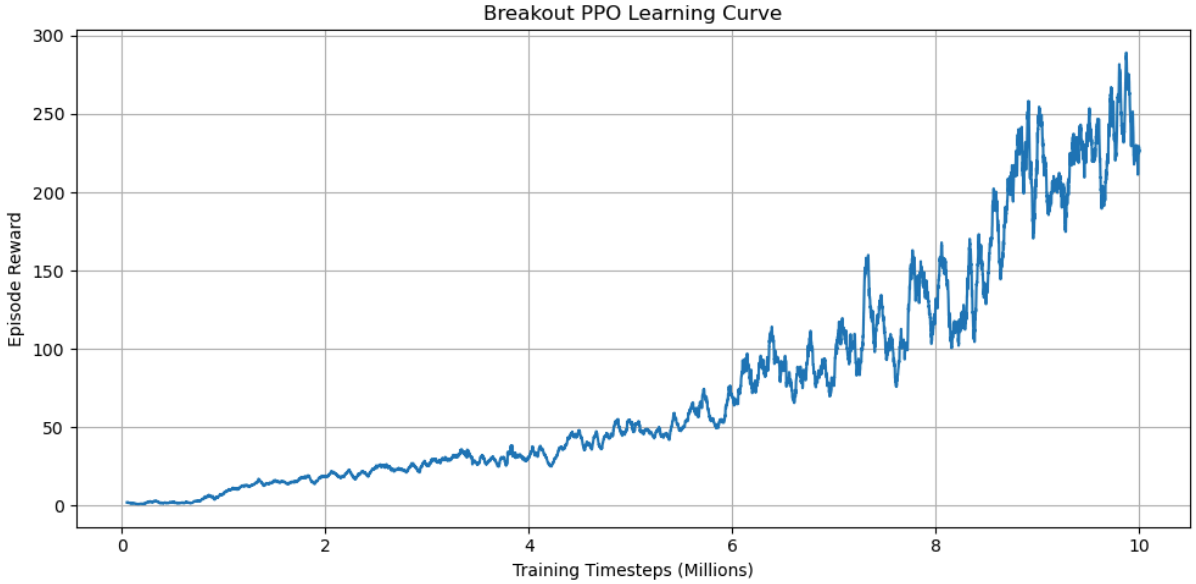


Figure 7. Breakout PPO Learning Curve

Cartpole

In the CartPole experiment, both DQN and PPO were trained using the same CartPole-v1 environment with a custom ANN architecture. The models were configured to train for 1 million timesteps. The training rewards were recorded and plotted to observe how the agents improved during training.

For DQN, Figure 8 shows that the episode reward was relatively low during the first 200,000 timesteps. After that, the reward gradually increased and reached above 100. Between 400,000 and 900,000 timesteps, the reward sometimes reached around 400 to 480, but it also dropped significantly several times. During the final 100,000 timesteps, the reward decreased to below 100. This shows that DQN was able to improve its ability to balance the pole, but its performance remained unstable throughout training.

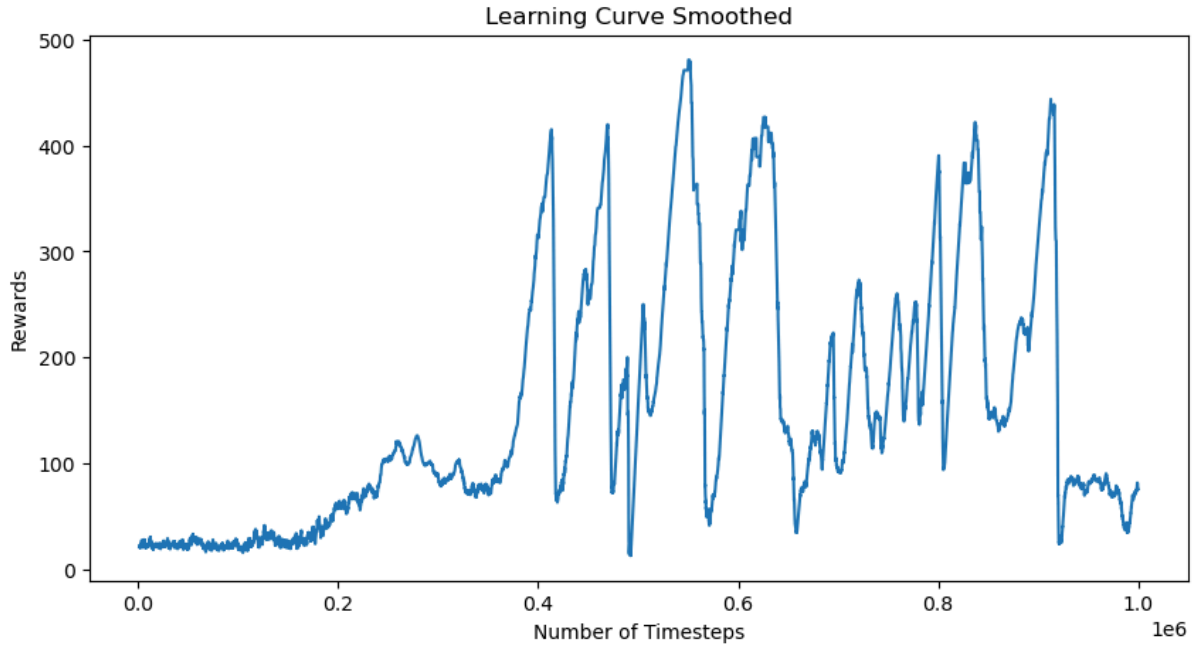


Figure 8. CartPole DQN Learning Curve

For PPO, Figure 9 shows that the episode reward increased rapidly during the first 100,000 timesteps and reached approximately 500. After that, the reward remained close to 500 for most of the training, although it occasionally dropped to lower values. During the final stage, the reward fluctuated again but recovered to around 470. This shows that PPO was able to learn how to balance the pole much faster than DQN and maintained relatively stable performance throughout most of the training.

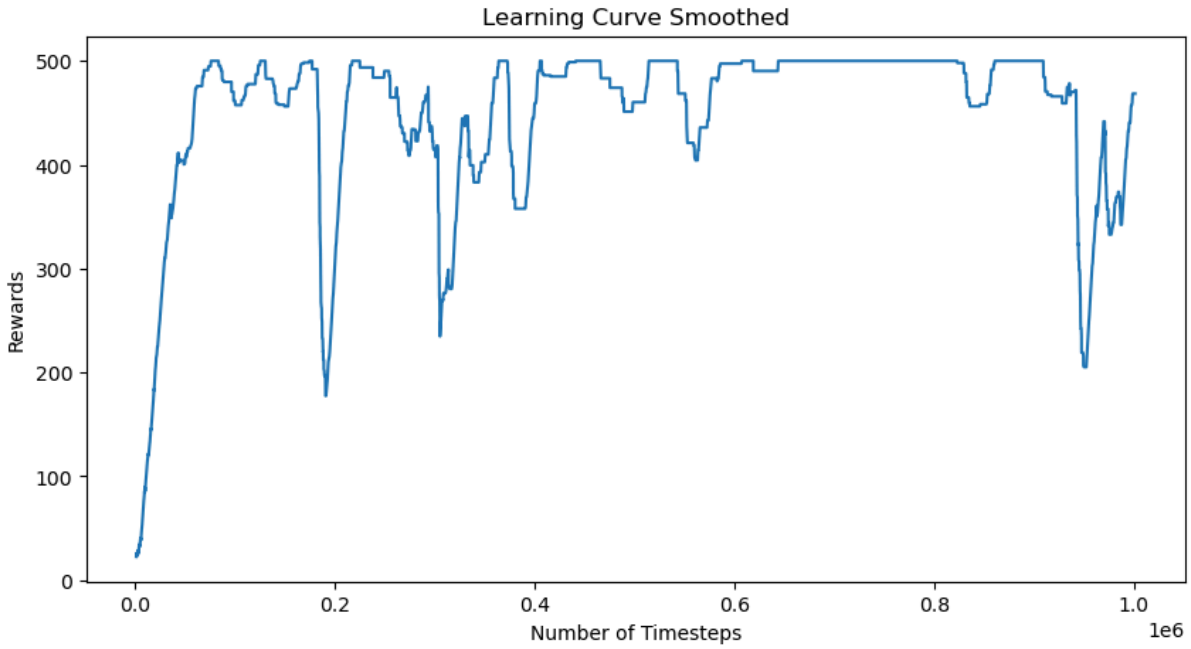


Figure 9. CartPole PPO Learning Curve

Pendulum

In the Pendulum experiment, Tabular Q-learning was trained using the Pendulum-v1 environment for 20,000 episodes. The continuous states and actions were discretized so that the agent could learn using a Q-table. The episode rewards and 100-episode moving average were recorded and plotted to observe how the agent improved during training.

Figure 10 shows that the average episode reward was around -1,200 during the early stages of training. After that, the reward gradually increased and reached approximately -500 at 5,000 episodes. The reward continued to improve and reached around -250 after 10,000 episodes. During the remaining episodes, the moving average stayed around -200 to -250, although individual episode rewards still fluctuated. This shows that the agent learned to control the pendulum more effectively.

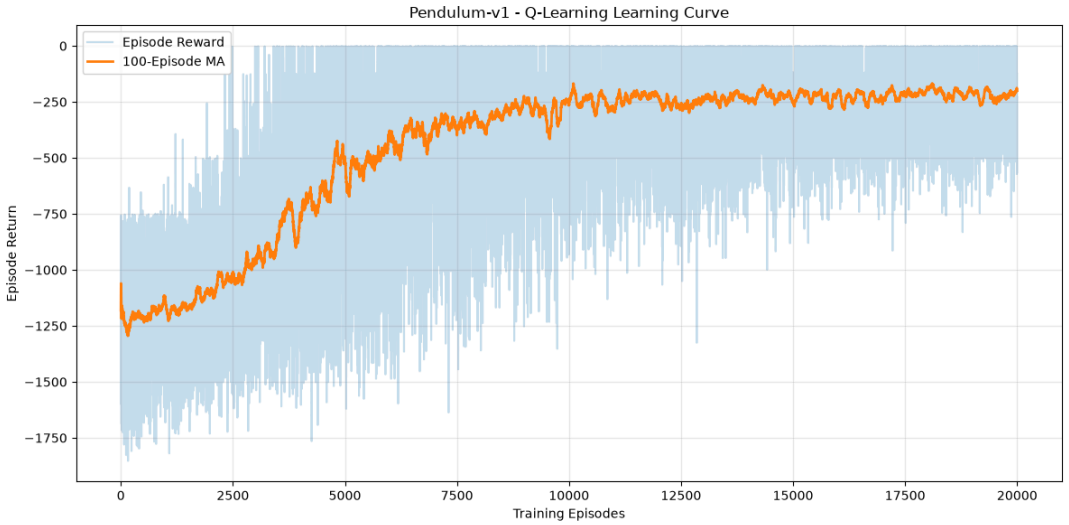


Figure 10. Pendulum Q-learning Learning Curve

Performance Evaluation

Breakout

The trained DQN and PPO models were evaluated separately over 20 episodes using deterministic actions. Both models used the same Breakout environment configuration and preprocessing settings to allow their performance to be compared. The average episode reward and standard deviation were used to measure the performance and consistency of each algorithm.

Metric	DQN	PPO
Evaluation Episodes	20	20
Average Episode Reward	329.10	262.70
Standard Deviation	21.59	110.16

Table 6. Breakout Performance Evaluation

The results from Table 6 show that DQN achieved a higher average reward of 329.10, compared with 262.70 for PPO. DQN also had a lower standard deviation of 21.59, while PPO had 110.16. This indicates that DQN performed better and produced more consistent results during evaluation. In contrast, PPO showed greater variation in episode rewards, meaning that its performance was less consistent across the 20 episodes.

These evaluation results are also consistent with the learning curves shown in Figures 6 and 7. Although PPO started learning earlier than DQN, its rewards fluctuated more during the later stages of training. In contrast, DQN showed a greater improvement towards the end of training and achieved a higher average reward during evaluation. This suggests that DQN was more effective for the Breakout task under the experimental settings used in this project.

Cartpole

The trained DQN and PPO models were evaluated separately over 20 episodes using deterministic actions. Both models used the same CartPole-v1 environment configuration to allow their performance to be compared. The average episode reward and standard deviation were used to measure the performance and consistency of each algorithm.

Metric	DQN	PPO
Evaluation Episodes	20	20
Average Episode Reward	81.70	500.00
Standard Deviation	10.00	0

Table 7. CartPole Performance Evaluation

The results from Table 7 show that PPO achieved the maximum average reward of 500.00, compared with 81.70 for DQN. PPO also had a standard deviation of 0.00, meaning that it achieved 500 rewards in all 20 evaluation episodes. In contrast, DQN had a standard deviation of 10.00 and a much lower average reward. This indicates that PPO was able to balance the pole for the full 500 steps consistently, while DQN could only maintain balance for a much shorter period.

Pendulum

The Q-learning agent was evaluated using the saved Q-tables at Episodes 1,000, 6,000, and 20,000. Each checkpoint was tested using the same initial state and greedy action selection without exploration. The episode reward was used to compare the agent's performance at different stages of training.

Checkpoint	Episode Reward
Episode 1,000	-967.64
Episode 6,000	-124.38
Episode 20,000	-125.90

Table 8. Pendulum Q-learning Performance Evaluation

The results from Table 8 show that the episode reward improved significantly from -967.64 at Episode 1,000 to -124.38 at Episode 6,000. The final checkpoint achieved a reward of -125.90, which was slightly lower than the result at Episode 6,000. This indicates that the agent learned a better control policy compared with the early stage of training. However, the performance did not improve further in this particular simulation after Episode 6,000.

Discussion

Breakout

The gameplay simulations were recorded at the beginning, middle, and end of training to observe how DQN and PPO improved their performance.

At the beginning of training, DQN achieved a score of 22, while PPO achieved a slightly higher score of 28. Both agents were still learning how to control the paddle and hit the ball. In both simulations, the game ended because the ball was lost. This shows that both algorithms had limited gameplay performance during the early stage of training.

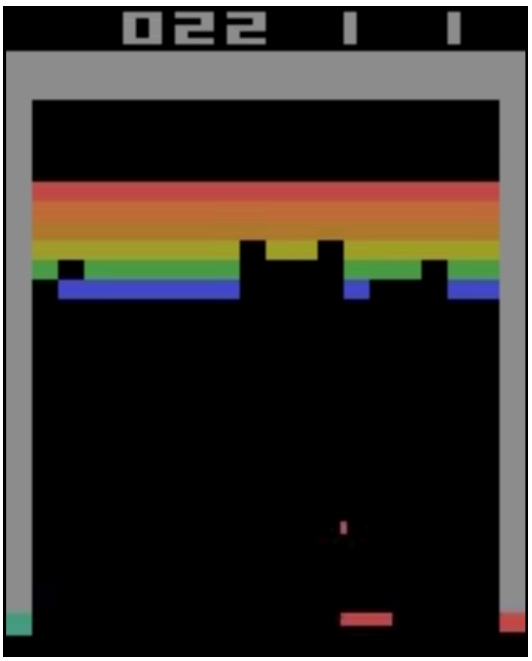


Figure 11. DQN Breakout Gameplay

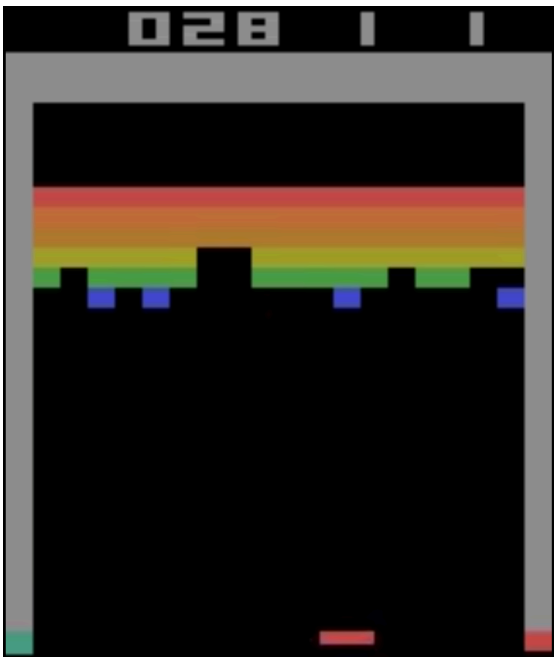


Figure 12. PPO Breakout Gameplay

During the middle stage of training, DQN achieved a score of 79, while PPO achieved 93. Both agents showed improvement compared with the beginning stage, indicating that they had learned to control the paddle more effectively and break more bricks. However, both games still ended when the ball was lost. PPO continued to achieve a slightly higher score than DQN at this stage.



Figure 13. DQN Breakout Gameplay



Figure 14. PPO Breakout Gameplay

At the end of training, DQN achieved a score of 347, while PPO achieved 277. Both agents showed significant improvement compared with the earlier stages. An interesting observation was that both agents learned to break the bricks along the sides of the screen, allowing the ball to move behind the remaining bricks and bounce between the bricks and

the upper wall. This helped the agents break more bricks and achieve higher scores. However, both games eventually ended when the ball was lost. DQN achieved a higher final score than PPO, which is also consistent with its higher average reward during performance evaluation.



Figure 15. DQN Breakout Gameplay



Figure 16. PPO Breakout Gameplay



Figure 17. DQN Breakout Gameplay



Figure 18. PPO Breakout Gameplay

Cartpole

At the beginning of training, the DQN agent was unable to keep the pole balanced on the cart. As a result, the cart moved towards the edge of the screen and eventually went out of view. In contrast, the PPO agent was able to keep the pole upright and maintain balance. This shows that PPO learned to control the cart and pole more effectively during the early stage of training.

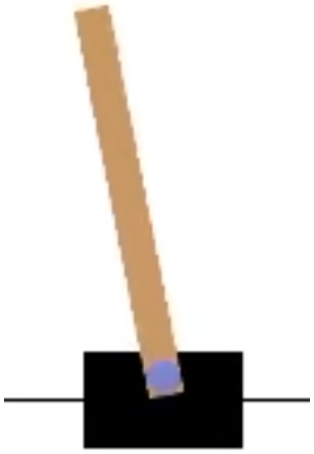


Figure 19. DQN CartPole Simulation

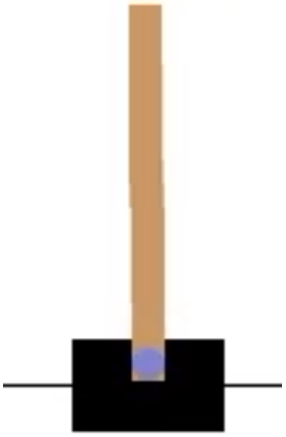


Figure 20. PPO CartPole Simulation

At the end of training, both DQN and PPO agents were able to keep the pole upright on the cart. This shows that both algorithms improved their ability to balance the pole as training progressed. However, the performance evaluation showed that PPO was more consistent, achieving an average reward of 500.00 compared with 81.70 for DQN. This indicates that although both agents could balance the pole in the recorded simulations, PPO was able to maintain balance for a longer period and performed more consistently across the evaluation episodes.

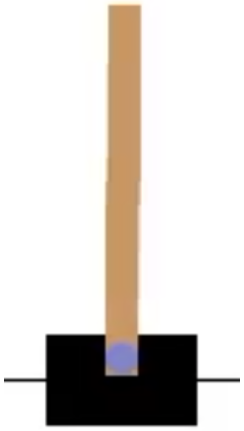


Figure 21. DQN CartPole Simulation

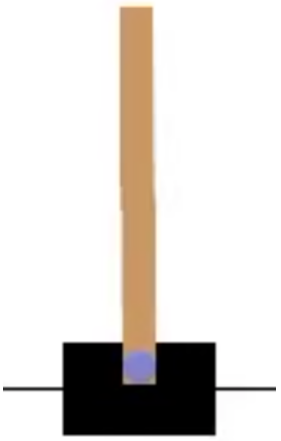


Figure 22. PPO CartPole Simulation

Pendulum

At the beginning of training, the Q-learning agent attempted to apply torque to control the pendulum. However, it was unable to keep the pendulum in an upright position. The pendulum continued to swing and could not maintain balance. This shows that the agent had not yet learned an effective control strategy during the early stage of training.

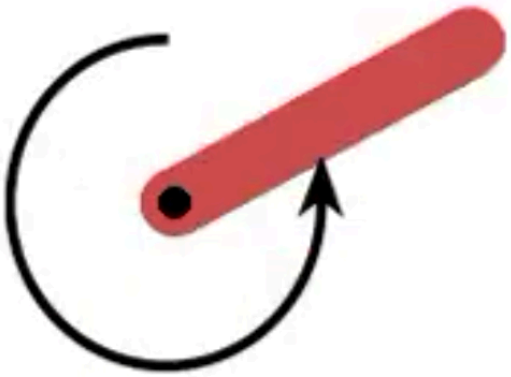


Figure 23. Pendulum Q-Learning Beginning Step Simulation

From the middle to the end of training, the Q-learning agent showed significant improvement. It learned to apply torque more effectively and was able to bring the pendulum into an upright position and maintain balance. This improvement is also supported by the simulation results, where the episode reward increased from -967.64 at Episode 1,000 to -124.38 at Episode 6,000 and -125.90 at Episode 20,000. However, the agent's performance was not completely consistent, as the episode rewards still fluctuated during training.

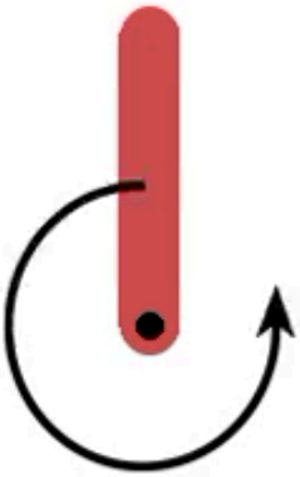


Figure 24. Pendulum Q-Learning Middle Step Simulation

Conclusion

In this assignment, I applied DQN, PPO, and Tabular Q-learning to three different reinforcement learning environments. From the experiments, I found that DQN performed better than PPO in Breakout, while PPO performed much better in CartPole. This shows that different algorithms have their own strengths and limitations, and their performance depends on the environment and training settings.

For Breakout, DQN achieved more consistent results, showing its strength in learning action values through experience replay. However, DQN took longer to improve during training. PPO showed faster improvement in the early stages, but its final performance was less consistent. In CartPole, PPO was able to learn a stable policy and maintain balance more effectively, while DQN struggled to achieve consistent results. This suggests that PPO can be effective for simpler control tasks, although its performance in Breakout shows that it does not always produce stable results. For Pendulum, Tabular Q-learning improved throughout training, showing that a simple Q-table can learn useful actions without a neural network. However, discretizing the continuous states and actions limits the precision of the agent's control.

Overall, these experiments helped me understand the strengths and limitations of value-based and policy-based reinforcement learning methods. I also learned that an algorithm that performs well in one environment may not perform equally well in another, and that selecting suitable training settings is important for achieving good performance.